

Министерство науки и высшего образования Российской
Федерации

Федеральное государственное автономное образовательное
учреждение
высшего образования

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО»

ОТЧЁТ

по домашнему заданию №3

по дисциплине «Бэкенд-разработка»

Тема: Тестирование API средствами Postman

Выполнил: Иванов В.С.

Группа: БР1.1

Вариант: Система бронирования столиков
в ресторанах

Проверил: Добряков Д.И.

Санкт-Петербург

2026

Содержание

1	Цель работы	2
2	Задание	2
3	Реализованный API	2
3.1	Реализованные эндпоинты	3
4	Тестовый сценарий	4
4.1	Структура коллекции	4
4.2	Шаги сценария	4
4.3	Пример теста (JavaScript)	6
5	Результаты тестирования	6
6	Вывод	7

1 Цель работы

Целью данного домашнего задания является реализация тестирования REST API приложения бронирования столиков в ресторанах средствами Postman. В рамках работы необходимо:

- разработать комплексный сценарий тестирования по основному процессу приложения;
- реализовать автоматические тесты внутри Postman с помощью скриптов на JavaScript;
- обеспечить автоматическую передачу данных (токен, идентификаторы) между запросами;
- охватить как позитивные, так и негативные тестовые случаи.

2 Задание

- Реализовать тестирование API средствами Postman.
- Написать тесты внутри Postman.
- Представить один комплексный сценарий по основному процессу приложения.

Основной сценарий для системы бронирования столиков:

*регистрация → вход → список ресторанов → фильтрация → просмотр рес
→ проверка доступности → бронирование → отзыв → отмена брони*

3 Реализованный API

В качестве тестируемого сервиса выступает REST API, разработанный в рамках лабораторной работы №1. Стек технологий:

- **Runtime:** Node.js

- **Framework:** Express.js
- **ORM:** TypeORM
- **База данных:** SQLite
- **Аутентификация:** JWT (Bearer токен)
- **Документация:** Swagger UI (/api-docs)

API запущен локально по адресу `http://localhost:3001`. Базовый префикс всех эндпоинтов: `/api/v1`.

3.1 Реализованные эндпоинты

Таблица 1: Эндпоинты REST API

Метод	Путь	Описание
POST	/auth/register	Регистрация пользователя
POST	/auth/login	Вход в систему, получение JWT
GET	/users/me	Профиль текущего пользователя
PATCH	/users/me	Обновление профиля
PUT	/users/me/password	Смена пароля
GET	/restaurants	Список ресторанов (фильтрация, сортировка)
GET	/restaurants/{id}	Детали ресторана
GET	/restaurants/{id}/tables	Столики ресторана
GET	/restaurants/{id}/availability	Доступность столиков
GET	/restaurants/{id}/reviews	Отзывы о ресторане
POST	/restaurants/{id}/reviews	Добавить отзыв
GET	/bookings	Список броней пользователя
POST	/bookings	Создать бронирование
GET	/bookings/{id}	Детали бронирования
DELETE	/bookings/{id}	Отменить бронирование

4 Тестовый сценарий

4.1 Структура коллекции

Postman-коллекция «**Restaurant Booking API**» содержит 18 запросов, сгруппированных в один сквозной сценарий. Коллекция использует переменные для автоматической передачи данных между запросами:

Таблица 2: Переменные коллекции

Переменная	Назначение
<code>base_url</code>	Базовый адрес API (<code>http://localhost:3001/api/v1</code>)
<code>access_token</code>	JWT токен, сохраняется после регистрации/логина
<code>restaurant_id</code>	ID первого ресторана из списка
<code>table_id</code>	ID первого столика выбранного ресторана
<code>booking_id</code>	ID созданного бронирования
<code>test_email</code>	Уникальный email (генерируется в pre-request скрипте)

4.2 Шаги сценария

- Регистрация нового пользователя** — `POST /auth/register`.
Pre-request скрипт генерирует уникальный email на основе timestamp.
После успешного ответа токен сохраняется в переменную `access_token`.
- Вход в систему** — `POST /auth/login`. Проверка что логин возвращает валидный JWT с теми же учётными данными.
- Профиль пользователя** — `GET /users/me`. Проверка авторизации через Bearer токен.
- Список ресторанов** — `GET /restaurants`. Сохранение `restaurant_id` из первого элемента.
- Фильтрация ресторанов** — `GET /restaurants?cuisine_type=русская`.
Проверка что все возвращённые рестораны соответствуют фильтру.

6. **Детали ресторана** — GET /restaurants/{restaurant_id}. Проверка полноты данных и наличия счётчиков.
7. **Список столиков** — GET /restaurants/{id}/tables. Сохранение table_id.
8. **Проверка доступности** — GET /restaurants/{id}/availability. Запрос с параметрами date, time, guests.
9. **Создание бронирования** — POST /bookings. Сохранение booking_id. Проверка статуса confirmed.
10. **Детали бронирования** — GET /bookings/{booking_id}. Проверка вложенных данных ресторана и столика.
11. **Список бронирований** — GET /bookings?status=confirmed. Проверка фильтрации по статусу.
12. **Добавление отзыва** — POST /restaurants/{id}/reviews. Оценка 5, комментарий. Проверка данных автора в ответе.
13. **Просмотр отзывов** — GET /restaurants/{id}/reviews. Проверка что отзыв появился в списке.
14. **Отмена бронирования** — DELETE /bookings/{booking_id}. Ожидаемый статус 204 No Content.
15. **Повторная отмена** — DELETE /bookings/{booking_id}. Негативный тест: ожидается 409 Conflict с кодом cannot_cancel_booking.
16. **Запрос без токена** — GET /users/me без заголовка Authorization. Ожидается 401 Unauthorized.
17. **Неверный пароль** — POST /auth/login с неправильным паролем. Ожидается 401 с кодом invalid_credentials.
18. **Несуществующий ресторан** — GET /restaurants/00000000-.... Ожидается 404 Not Found.

4.3 Пример теста (JavaScript)

Ниже приведён пример тестового скрипта для запроса «Создание бронирования»:

```
1 pm.test('                201 -                ',  
    function () {  
2     pm.response.to.have.status(201);  
3 });  
4  
5 pm.test('                = confirmed', function () {  
6     pm.expect(pm.response.json().status).to.equal('confirmed');  
7 });  
8  
9 pm.test('                ', function () {  
10     var json = pm.response.json();  
11     pm.expect(json).to.have.property('restaurant');  
12     pm.expect(json).to.have.property('table');  
13 });  
14  
15 //                booking_id  
16 var json = pm.response.json();  
17 if (json.id) {  
18     pm.collectionVariables.set('booking_id', json.id);  
19 }
```

Листинг 1: Тест для POST /bookings

5 Результаты тестирования

Коллекция запущена через Collection Runner Postman (Run collection).

Таблица 3: Итоги прогона коллекции

Показатель	Значение
Всего запросов	18
Всего тестов	55
Passed	55
Failed	0
Errors	0
Skipped	0
Среднее время ответа	17 мс
Общая длительность	827 мс

Все 55 проверок прошли успешно. Как позитивные сценарии (регистрация, бронирование, отзыв), так и негативные (401, 404, 409) возвращают ожидаемые коды и тела ответов.

6 Вывод

В ходе выполнения домашнего задания был реализован комплексный сценарий тестирования REST API системы бронирования столиков в ресторанах средствами Postman. Сценарий покрывает полный жизненный цикл основного бизнес-процесса: от регистрации пользователя до создания и отмены бронирования.

Ключевые результаты:

- разработана Postman-коллекция из 18 запросов с 55 автоматическими проверками;
- реализована автоматическая передача токенов и идентификаторов между шагами через переменные коллекции;
- все тесты успешно пройдены (**55/55 Passed, 0 Failed**);
- покрыты негативные сценарии: попытка доступа без токена (401), повторная отмена брони (409), запрос несуществующего ресурса (404).